

Linguagens Formais e Autômatos

MeteoRegex — analisador de telemetria de estações meteorológicas

Trabalho do 1º Bimestre — Expressões Regulares e AFNe

Relatório Técnico

[NOME COMPLETO DO INTEGRANTE 1]
[NOME COMPLETO DO INTEGRANTE 2]
[NOME COMPLETO DO INTEGRANTE 3]
[NOME COMPLETO DO INTEGRANTE 4]

Linguagem: Python 3.10+ • 6 Expressões Regulares • 6 AFNe • 78 testes automatizados
Entrega: 02/10/2026

1. Resumo

Este relatório descreve o **MeteoRegex**, uma aplicação de linha de comando escrita em Python que valida, extrai e resume registros de telemetria de uma rede de estações meteorológicas automáticas. O programa aplica **seis Expressões Regulares** — uma para cada campo do registro — e implementa, no mesmo código, os **AFNε equivalentes**, construídos segundo as regras de Thompson estudadas na disciplina. Uma bateria de 78 testes automatizados verifica não apenas o comportamento esperado das expressões, mas também a **equivalência entre cada Expressão Regular e seu autômato**, em todas as cadeias de teste, em 12 000 cadeias aleatórias e em 7 200 mutações de cadeias válidas.

2. Problema e justificativa

Estações meteorológicas automáticas transmitem, a cada leitura, uma linha de texto com campos heterogêneos: identificador da estação, instante da coleta, posição geográfica, leituras de sensores com unidade, versão do firmware embarcado e, eventualmente, um alerta hidrometeorológico. Falhas de transmissão, sensores descalibrados e firmwares divergentes produzem linhas truncadas ou mal formadas. Antes de qualquer análise climatológica é preciso separar, de forma automática e auditável, o que é sintaticamente válido do que não é — e indicar *por que* cada linha foi rejeitada.

O problema é adequado ao conteúdo da disciplina porque cada campo do registro é, isoladamente, uma **linguagem regular**: sua estrutura pode ser descrita por união, concatenação e fechos, e reconhecida por um autômato finito. Ao mesmo tempo, o trabalho evidencia com clareza os **limites** do modelo regular: nenhuma das expressões consegue decidir se uma data existe no calendário ou se uma temperatura é fisicamente plausível, o que exige uma camada de verificação semântica posterior.

2.1 Entradas, processamento e saídas

Etapa	Descrição
Entrada	arquivo .log de telemetria, uma linha digitada pelo usuário ou uma cadeia avulsa a ser testada contra uma Expressão Regular específica.
Processamento	separação dos campos, validação de cada campo com re.fullmatch, extração dos dados, verificações semânticas complementares (calendário, faixas geográficas, faixas físicas, catálogo de alertas) e cálculo de estatísticas.
Saída	relatório em texto com resumo, estatísticas por grandeza, alertas por nível, firmwares em operação, avisos semânticos e a lista dos registros rejeitados com a mensagem exata do motivo.

2.2 Formato do registro

```
codigo | carimbo | coordenadas | leituras | firmware [ | alerta ]
ER-01   ER-02   ER-04       ER-03   ER-06       ER-05

PA-BEL-004A|2026-09-21T14:35:02Z|-1.455833,-48.503889|TEMP=+27.4C;UMID=85%;PLUV=12.5mm|fw-v3.11.2-beta.4|!ALERTA:NIVEL2:CHUVA_FORTE
```

O sexto campo é opcional; o quarto pode conter várias leituras separadas por ponto e vírgula. Linhas em branco e linhas iniciadas por # são tratadas como comentários.

3. Fundamentação teórica

Uma **Expressão Regular** sobre um alfabeto finito Σ é definida indutivamente a partir de $\emptyset$, ϵ e dos símbolos de Σ , combinados por união ($r \mid s$), concatenação (rs) e fecho de Kleene (r^*). Os demais operadores usados neste trabalho são abreviações: $r^+ = rr^*$, $r? = (r \mid \epsilon)$, $r\{m\}$ é a concatenação de m cópias e $r\{m,n\}$ é a união de $r\{m\}$, ..., $r\{n\}$. As classes $[abc]$ e $[0-9]$ abreviam uniões finitas de símbolos.

Um **AFNε** é a quintupla $A = (Q, \Sigma, \delta, q_0, F)$, em que δ leva um par (estado, símbolo $u \in \{\epsilon\}$) a um *conjunto* de estados. O movimento vazio permite mudar de estado sem consumir símbolo algum da entrada, o que é exatamente o que torna direta a construção de Thompson: para cada operador da Expressão Regular existe um fragmento de autômato com uma entrada e uma saída, e os fragmentos são conectados por transições ϵ .

Pelo **teorema de Kleene**, uma linguagem é regular se, e somente se, é descrita por uma Expressão Regular e reconhecida por um autômato finito. Neste trabalho essa equivalência não é apenas citada: ela é *verificada experimentalmente*, pois o programa executa a Expressão Regular no motor re e simula o AFNε correspondente, comparando os vereditos sobre milhares de cadeias.

3.1 Construções elementares usadas

Operador	Fragmento construído
concatenação rs	fragmentos em série: a saída de r é a entrada de s .
união $r \mid s$	transições ϵ do estado de escolha para a entrada de cada ramo e transições ϵ das saídas dos ramos para um estado de junção.

fecho positivo r^+	uma ocorrência obrigatória e uma transição ϵ de retorno ao estado anterior.
fecho de Kleene r^*	como r^+ , acrescido de uma transição ϵ de desvio (ramo da palavra vazia).
opcionalidade $r?$	transição ϵ que salta o fragmento, representando o ramo ϵ da união $(r \mid \epsilon)$.
repetição $r\{m,n\}$	m cópias em série e $(n-m)$ cópias adicionais, cada uma com saída por ϵ para o estado final do bloco.

Para manter os diagramas legíveis, as transições podem ser rotuladas por classes escritas entre colchetes angulares — $\langle M \rangle = [A-Z]$, $\langle D \rangle = [0-9]$, $\langle S \rangle = [+ -]$ e $\langle N \rangle = [1-3]$. Uma transição rotulada por uma classe é abreviação da união das transições sobre cada símbolo da classe, exatamente como $[A-Z]$ abrevia $(A \mid B \mid \dots \mid Z)$. A notação angular evita qualquer confusão com símbolos literais (a letra N de “NIVEL”, por exemplo, é literal).

4. Arquitetura da aplicação

A linguagem escolhida foi **Python 3.10+**, cujo motor de Expressões Regulares (módulo `re`) oferece `fullmatch`, isto é, casamento da cadeia inteira — o que corresponde exatamente à pergunta “esta cadeia pertence à linguagem?”. A aplicação e os testes usam somente a biblioteca padrão.

Módulo	Responsabilidade
analizador/expressoes.py	declara as seis Expressões Regulares como constantes e as fichas de documentação (alfabeto, linguagem, ER formal, equivalências, testes, limitações). É a <i>fonte única de verdade</i> do projeto.
analizador/automatos.py	construtor de AFN ϵ ao estilo Thompson, simulador com ϵ -fecho e a definição dos seis autômatos.
analizador/processador.py	separa os campos, aplica as ER, extrai os dados e executa as verificações semânticas; calcula as estatísticas.
analizador/relatorio.py	formatação de toda a saída em texto.
analizador/cli.py	argumentos de linha de comando e menu interativo, com mensagens claras para toda entrada inválida.
ferramentas/	geradores dos diagramas SVG, da documentação Markdown, deste relatório e da apresentação — todos alimentados pelo próprio código.

4.1 Coerência entre expressão, código, testes e autômato

Cada padrão é escrito **uma única vez**, como constante de string bruta. Essa mesma constante é usada pelo validador, pela ficha exibida na tela, pela documentação em Markdown, por este relatório e pelos slides. Os diagramas, por sua vez, são desenhados a partir da mesma estrutura de dados que o simulador percorre. Assim, a exigência do enunciado — “a expressão formal, a expressão apresentada nos slides, o padrão implementado no código, os testes e o AFN ϵ deverão representar a mesma linguagem” — é garantida por construção, e não por conferência manual.

4.2 Tratamento de entradas vazias e inválidas

- arquivo inexistente, vazio, ilegível ou diretório informado no lugar de um arquivo: mensagem [ERRO DE ENTRADA] e código de retorno 2;
- conteúdo apenas com linhas em branco ou comentários: mensagem específica;
- linha vazia, número de campos diferente de 5 ou 6, campo opcional presente porém vazio: registro rejeitado com o motivo;
- cadeia vazia (ϵ) na validação avulsa: aviso explicando que nenhuma das seis linguagens contém a palavra vazia;
- identificador de ER inexistente ou opção de menu inválida: mensagem indicando as opções válidas;
- nenhuma dessas situações interrompe o programa com rastreamento de exceção.

5. As seis Expressões Regulares

Cada ficha abaixo segue o roteiro obrigatório do guia de sintaxe: identificação, alfabeto, linguagem, Expressão Regular formal, sintaxe exatamente como aparece no código, equivalência dos atalhos, AFNε, testes com caso-limite e comportamento observado.

ER	Nome	Campo	Estados	Transições	ε
ER-01	Código de estação meteorológica	campo 1 do registro de telemetria	18	19	5
ER-02	Carimbo temporal ISO-8601	campo 2 do registro de telemetria	37	41	11
ER-03	Leitura de sensor com unidade	campo 4 do registro (itens separados por ';')	30	38	19
ER-04	Par de coordenadas geodésicas decimais	campo 3 do registro de telemetria	30	39	16
ER-05	Alerta hidrometeorológico	campo 6 (opcional) do registro de telemetria	21	23	5
ER-06	Versão de firmware da estação	campo 5 do registro de telemetria	30	36	13

ER-01 — Código de estação meteorológica

Finalidade. Validar o identificador da estação que originou o registro e permitir agrupar as leituras por estação e por sub-estação.

Aplicada em. campo 1 do registro de telemetria

Alfabeto (Σ)

$\Sigma_1 = M \cup D \cup \{ '-', '.' \}$, com $M = \{A, B, \dots, Z\}$ e $D = \{0, 1, \dots, 9\}$

Linguagem reconhecida (L)

Cadeias formadas por duas letras maiúsculas (sigla da UF), um hífen, três letras maiúsculas (código do município), um hífen, três dígitos (número sequencial da estação) e uma letra maiúscula (revisão do hardware), seguidas opcionalmente de um ponto e de um ou dois dígitos que identificam a sub-estação.

Expressão Regular na notação formal

$M M - M M M - D D D M (. D (D | \epsilon) | \epsilon)$
forma abreviada: $M\{2\} - M\{3\} - D\{3\} M (. D\{1,2\}) ?$

Sintaxe exatamente como no código-fonte (Python)

`r"[A-Z]{2}-[A-Z]{3}-[0-9]{3}[A-Z](?:\.[0-9]{1,2})?"`

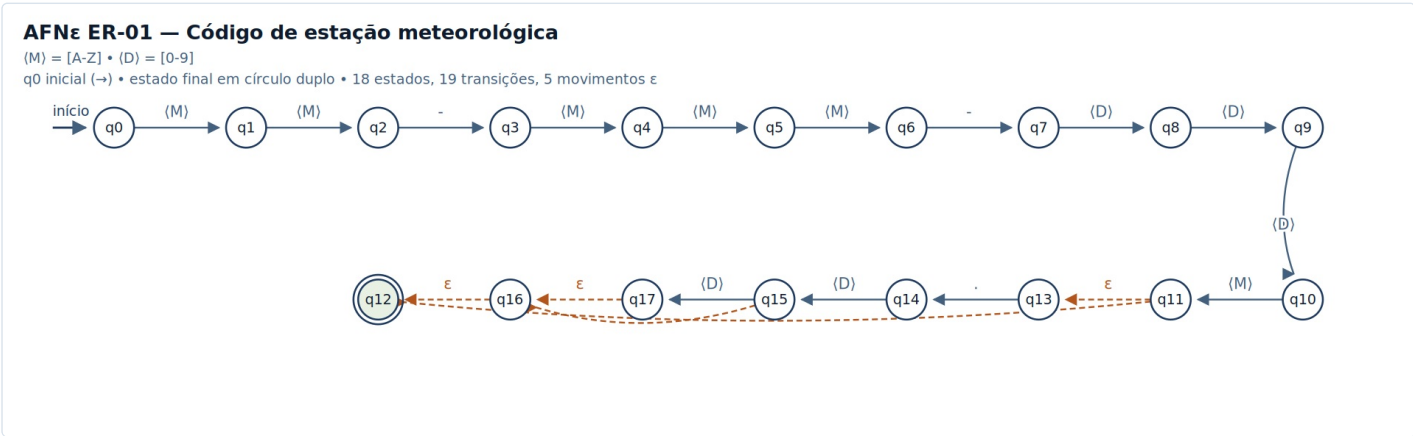
Equivalência entre atalhos e operadores formais

- $[A-Z] \equiv (A \mid B \mid \dots \mid Z) =$ classe M
- $[0-9] \equiv (0 \mid 1 \mid \dots \mid 9) =$ classe D
- $r\{2\} \equiv r r$ (concatenação de 2 cópias)
- $r\{3\} \equiv r r r$ (concatenação de 3 cópias)
- $r\{1,2\} \equiv (r \mid r r)$
- $(?:r)? \equiv (r \mid \epsilon)$
- `\.` $\equiv$ símbolo literal '.' (o escape remove o significado especial)

Operadores utilizados

- concatenação: impõe a ordem rígida dos blocos do identificador;
- classe/intervalo: abreviação da união dos símbolos do alfabeto;
- repetição exata $\{m\}$: concatenação de m cópias;
- repetição limitada $\{1,2\}$: união de uma e de duas cópias;
- opcionalidade $?$: união com a palavra vazia ϵ (sufixo de sub-estação).

AFNε correspondente



Estado inicial: q0 • estado(s) final(is): q12 • 18 estados, 19 transições, 5 movimentos vazios (arestas tracejadas). Descrição formal completa em docs/afne_formal.md.

Testes

#	Cadeia aceita	Cadeia rejeitada
1	PA-BEL-004A	pa-bel-004a
2	PA-ANA-127B	PA-BE-004A
3	AM-MAO-001A	PA-BEL-04A

4	PA-BEL-004A.2	PA-BEL-004
5	RJ-RIO-999Z.15	PA-BEL-004A.
6	SP-SAO-000A.1	PA-BEL-004A.123

Caso-limite. "PA-BEL-004A." — prefixo válido seguido do ponto sem nenhum dígito: o fecho opcional exige que, havendo o ponto, exista ao menos um dígito. Caso-limite oposto: "PA-BEL-004A" (sufixo ausente, ramo ε) é aceito.

Resultado observado e limitações. A ER valida apenas a FORMA do código. Não verifica se a sigla é uma UF existente nem se a estação está cadastrada: "ZZ-XXX-000A" é formalmente aceita. Essa verificação semântica é feita depois, por consulta à lista de estações conhecidas.

ER-02 — Carimbo temporal ISO-8601

Finalidade. Validar o instante da coleta, com fração de segundo e fuso horário opcionais, permitindo ordenar cronologicamente os registros.

Aplicada em. campo 2 do registro de telemetria

Alfabeto (Σ)

$\Sigma_2 = D \cup \{ '-', 'T', ':', '.', 'Z', '+' \}$, com $D = \{0, 1, \dots, 9\}$

Linguagem reconhecida (L)

Cadeias no formato AAAA-MM-DDThh:mm:ss, opcionalmente seguidas de um ponto e de 1 a 3 dígitos (milissegundos) e, opcionalmente, de um indicador de fuso: a letra Z (UTC) ou um sinal seguido de hh:mm.

Expressão Regular na notação formal

$D\{4\} - D\{2\} - D\{2\} T D\{2\} : D\{2\} : D\{2\} (. D (D (D | \epsilon) | \epsilon) | \epsilon) (Z | (+ | -) D\{2\} : D\{2\} | \epsilon)$

Sintaxe exatamente como no código-fonte (Python)

`r"[0-9]{4}-[0-9]{2}-[0-9]{2}T[0-9]{2}:[0-9]{2}:[0-9]{2}(?:\.[0-9]{1,3})?(?:Z|[-+][0-9]{2}:[0-9]{2})?"`

Equivalência entre atalhos e operadores formais

- $[0-9] \equiv (0 | 1 | \dots | 9) = \text{classe } D$
- $r\{4\} \equiv rrrr$
- $r\{1,3\} \equiv (r | rr | rrr)$
- $[+-] \equiv (+ | -)$ — dentro da classe o '-' está na borda e é literal
- $(?:r)? \equiv (r | \epsilon)$
- $(?:a|b) \equiv \text{união } a | b$

Operadores utilizados

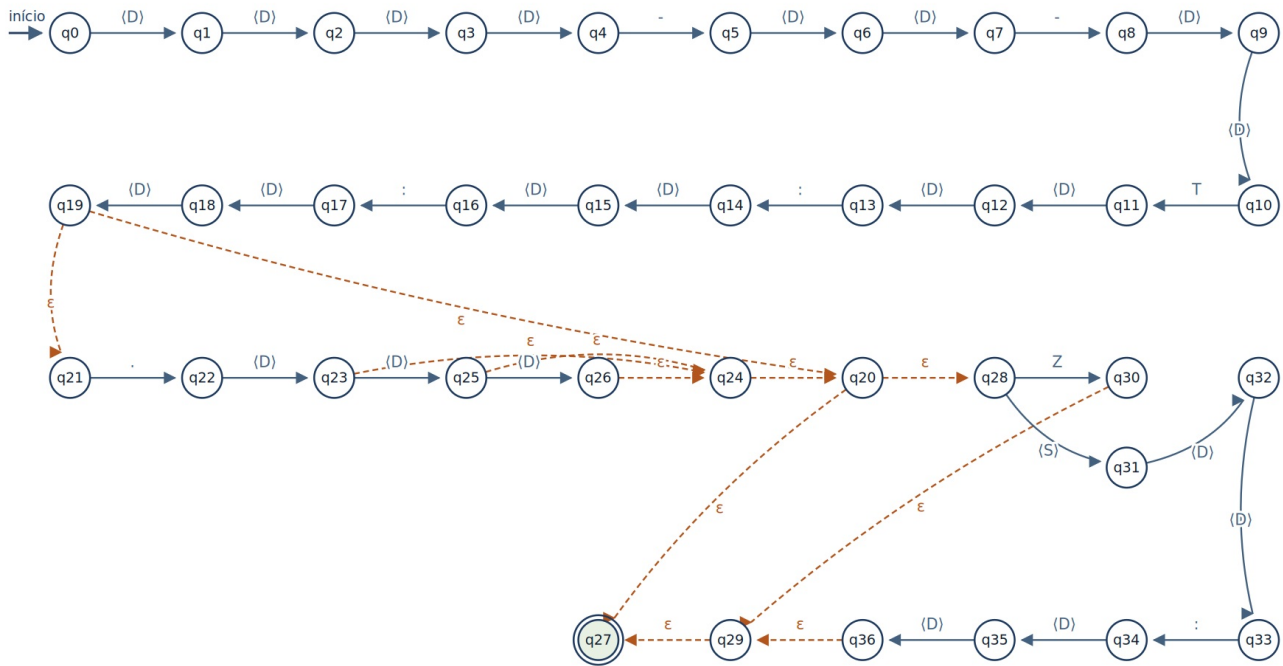
- concatenação: fixa a ordem data $\rightarrow$ 'T' $\rightarrow$ hora;
- repetição exata $\{m\}$: blocos de tamanho fixo (ano, mês, dia, hora...);
- repetição limitada $\{1,3\}$: fração de segundo de 1 a 3 dígitos;
- união $|$: escolhe entre fuso 'Z' e deslocamento $\pm$ hh:mm;
- opcionalidade $?$: dois sufixos independentes podem ser ϵ .

AFNε correspondente

AFNε ER-02 — Carimbo temporal ISO-8601

(D) = [0-9] • (S) = [+·-]

q0 inicial (→) • estado final em círculo duplo • 37 estados, 41 transições, 11 movimentos ε



Estado inicial: q0 • estado(s) final(is): q27 • 37 estados, 41 transições, 11 movimentos vazios (arestas tracejadas). Descrição formal completa em docs/afne_formal.md.

Testes

#	Cadeia aceita	Cadeia rejeitada
1	2026-09-21T14:35:02	2026-09-21 14:35:02
2	2026-09-21T14:35:02Z	26-09-21T14:35:02
3	2026-09-21T14:35:02.5	2026-09-21T14:35
4	2026-09-21T14:35:02.123-03:00	2026-09-21T14:35:02.
5	2026-01-01T00:00:00+00:00	2026-09-21T14:35:02.1234
6	1999-12-31T23:59:59.999Z	2026-09-21T14:35:02-0300

Caso-limite. "2026-09-21T14:35:02." — a parte obrigatória está completa, mas o ponto abre uma fração vazia; como {1,3} exige pelo menos um dígito, a cadeia é rejeitada. O caso-limite aceito correspondente é "2026-09-21T14:35:02" (ambos os sufixos opcionais em ε).

Resultado observado e limitações. A ER reconhece a ESTRUTURA do carimbo, não a validade do calendário: "2026-13-45T99:99:99" é aceita pela ER. Datas impossíveis são descartadas na etapa seguinte, por conversão com datetime.

ER-03 — Leitura de sensor com unidade

Finalidade. Validar e extrair cada par grandeza=valor+unidade produzido pelos sensores da estação, base de todo o cálculo estatístico do programa.

Aplicada em. campo 4 do registro (itens separados por ';')

Alfabeto (Σ)

$\Sigma_3 = M \cup D \cup \{ '=', '+', '-', '.', '%', 'm', 'h', 'P', 'a', '/', 's', 'C' \}$

Linguagem reconhecida (L)

Cadeias compostas por uma sigla de 3 ou 4 letras maiúsculas, o símbolo '=', um número decimal com sinal opcional e parte fracionária opcional, e uma unidade pertencente a $\{ C, \%, \text{mm}, \text{hPa}, \text{m/s} \}$.

Expressão Regular na notação formal

$M M M (M | \epsilon) = (+ | - | \epsilon) D D^* (. D D^* | \epsilon) (C | \% | m m | h P a | m / s)$

Sintaxe exatamente como no código-fonte (Python)

`r"[A-Z]{3,4}=[+-]?[0-9]+(?:\.[0-9]+)?(?:C|mm|hPa|m/s)"`

Equivalência entre atalhos e operadores formais

- $[A-Z]\{3,4\} \equiv M M M (M | \epsilon)$
- $[+-]? \equiv (+ | - | \epsilon)$
- $[0-9]^+ \equiv D D^*$ (fecho positivo = uma cópia seguida do fecho de Kleene)
- $(?:\.[0-9]^+)? \equiv (. D D^* | \epsilon)$
- $(?:C|mm|hPa|m/s) \equiv$ união de cinco cadeias literais

Operadores utilizados

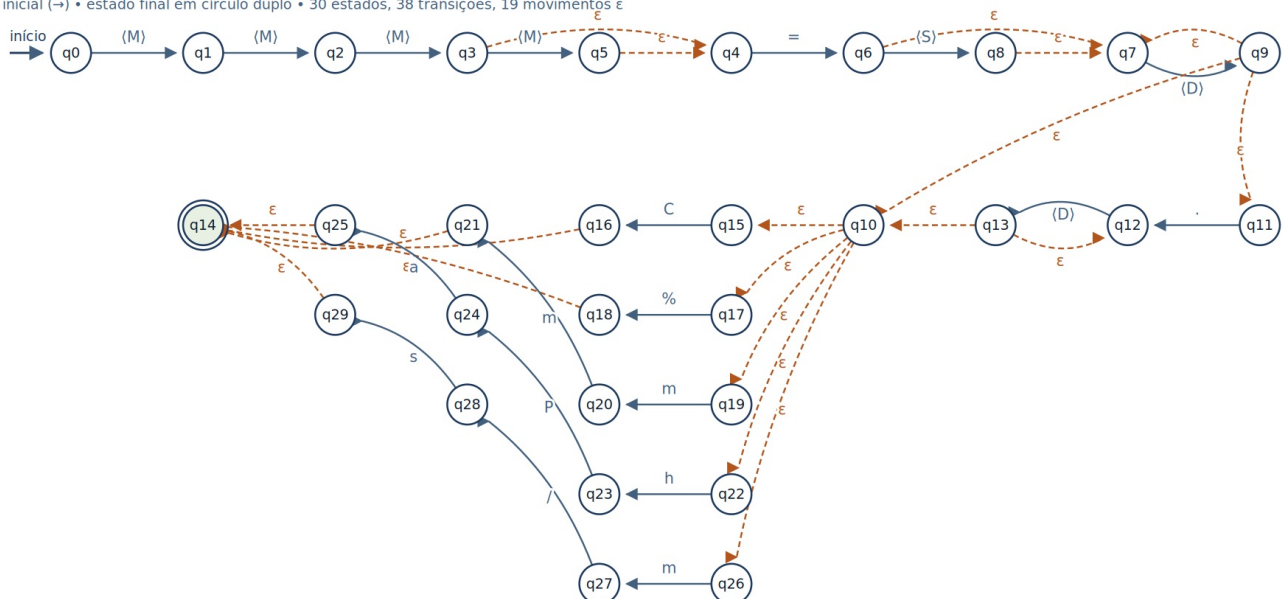
- fecho positivo +: a parte inteira tem pelo menos um dígito;
- fecho de Kleene *: implícito na expansão de + ($D D^*$);
- opcionalidade ?: sinal e parte fracionária podem ser ϵ ;
- união |: escolhe a unidade de medida;
- concatenação: amarra sigla, '=', número e unidade.

AFN ϵ correspondente

AFN ϵ ER-03 — Leitura de sensor com unidade

$\langle M \rangle = [A-Z]$ • $\langle S \rangle = [+ -]$ • $\langle D \rangle = [0-9]$

q0 inicial ($\rightarrow$) • estado final em círculo duplo • 30 estados, 38 transições, 19 movimentos ϵ



Estado inicial: q0 • estado(s) final(is): q14 • 30 estados, 38 transições, 19 movimentos vazios (arestas tracejadas). Descrição formal completa em docs/afne_formal.md.

Testes

#	Cadeia aceita	Cadeia rejeitada
1	TEMP=+27.4C	TEMP=27.4
2	UMID=85%	TE=27C
3	PLUV=0.0mm	TEMPER=27C
4	PRES=1012.75hPa	TEMP=.5C
5	VENT=3.2m/s	TEMP=27.C
6	TEMP=-0C	temp=27C

Caso-limite. "TEMP=.5C" — número sem parte inteira: o fecho positivo [0-9]+ exige ao menos um dígito antes do ponto, logo a cadeia é rejeitada. O caso-limite aceito é "TEMP=-0C", número mínimo com sinal e sem parte fracionária.

Resultado observado e limitações. A ER não impõe faixa física: "TEMP=+999.9C" é aceita. A checagem de plausibilidade (faixas por grandeza) é feita no módulo processador, que sinaliza a leitura como suspeita sem invalidar o registro.

ER-04 — Par de coordenadas geodésicas decimais

Finalidade. Validar a posição informada pela estação (latitude e longitude em graus decimais com sinal), usada no agrupamento geográfico do relatório.

Aplicada em. campo 3 do registro de telemetria

Alfabeto (Σ)

$$\Sigma_4 = D \cup \{ '+', '-', '.', ', ' \}, \text{ com } D = \{ 0, 1, \dots, 9 \}$$

Linguagem reconhecida (L)

Dois números decimais separados por vírgula; cada número possui sinal opcional, de 1 a 3 dígitos inteiros, ponto obrigatório e de 4 a 6 casas decimais.

Expressão Regular na notação formal

N , N, onde $N = (+ \mid - \mid \varepsilon) D (D (D \mid \varepsilon) \mid \varepsilon) . D D D D (D (D \mid \varepsilon) \mid \varepsilon)$
 forma abreviada: $N = (+ \mid -)? D\{1,3\} . D\{4,6\}$

Sintaxe exatamente como no código-fonte (Python)

```
r"[+-]?[0-9]{1,3}\.[0-9]{4,6},[+-]?[0-9]{1,3}\.[0-9]{4,6}"
```

Equivalência entre atalhos e operadores formais

- $[+ - ?] \equiv (+ \mid - \mid \varepsilon)$
- $[0-9]\{1,3\} \equiv (D \mid D D \mid D D D)$
- $[0-9]\{4,6\} \equiv (D\{4\} \mid D\{5\} \mid D\{6\})$
- $\backslash. \equiv$ símbolo literal $'.'$
- $'.' \equiv$ símbolo literal $'.'$

Operadores utilizados

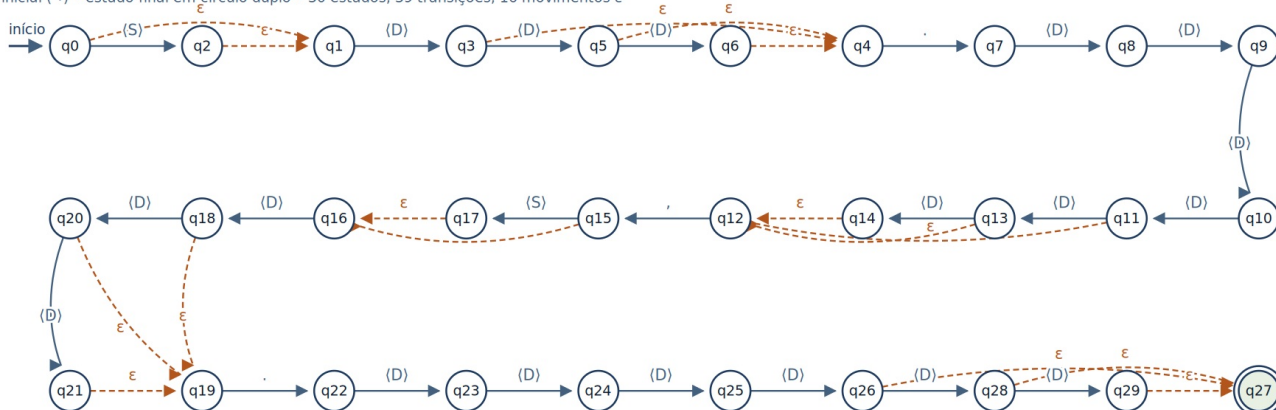
- repetição limitada {m,n}: união das repetições de m até n cópias;
- opcionalidade?: o sinal pode ser ϵ (coordenada positiva);
- concatenação: latitude, vírgula e longitude, nessa ordem;
- escape \. : garante que o ponto seja símbolo do alfabeto e não o curinga do motor.

AFNε corrispondente

AFNε ER-04 — Par de coordenadas geodésicas decimais

$$\langle S \rangle = [+ -] \cdot \langle D \rangle = [0 - 9]$$

q0 inicial (→) • estado final em círculo duplo • 30 estados, 39 transições, 16 movimentos e



Estado inicial: q0 • estado(s) final(is): q27 • 30 estados, 39 transições, 16 movimentos vazios (arestas tracejadas). Descrição formal completa em docs/afne_formal.md.

Testes

#	Cadeia aceita	Cadeia rejeitada
1	-1.455833,-48.503889	-1.455833
2	+1.4558,-48.5038	-1.45,-48.50

3	0.0000,0.0000	-1455833,-48503889
4	-23.550520,-46.633308	-1.455833;-48.503889
5	90.000000,180.000000	-1.4558333,-48.5038
6	-1.4558,48.5038	-1.4558, -48.5038

Caso-limite. "-1.45,-48.50" — coordenada plausível, porém com 2 casas decimais: a repetição {4,6} exige no mínimo 4, e a cadeia é rejeitada. O caso-limite aceito é "0.0000,0.0000" (mínimo de dígitos inteiros e de casas decimais, sem sinal).

Resultado observado e limitações. A ER não restringe a faixa geográfica: "999.0000,999.0000" seria aceita se tivesse 3 dígitos inteiros. A validação de faixa ($|lat| \leq 90$, $|lon| \leq 180$) é numérica e ocorre após o reconhecimento.

ER-05 — Alerta hidrometeorológico

Finalidade. Reconhecer o campo opcional de alerta, extraindo o nível de severidade e o código do evento para o painel de ocorrências.

Aplicada em. campo 6 (opcional) do registro de telemetria

Alfabeto (Σ)

$\Sigma_5 = M \cup \{ '!', ':', '_', '1', '2', '3' \}$, com $M = \{A, \dots, Z\}$

Linguagem reconhecida (L)

Cadeias iniciadas por "!ALERTA:NIVEL", seguidas de um dígito de 1 a 3, de ':' e de um código formado por uma ou mais palavras de letras maiúsculas separadas por sublinhado.

Expressão Regular na notação formal

`!ALERTA:NIVEL(1|2|3):P(_P)*`, onde $P = M M^*$

Sintaxe exatamente como no código-fonte (Python)

`r"!ALERTA:NIVEL[1-3]:[A-Z]+(?:_[A-Z]+)*"`

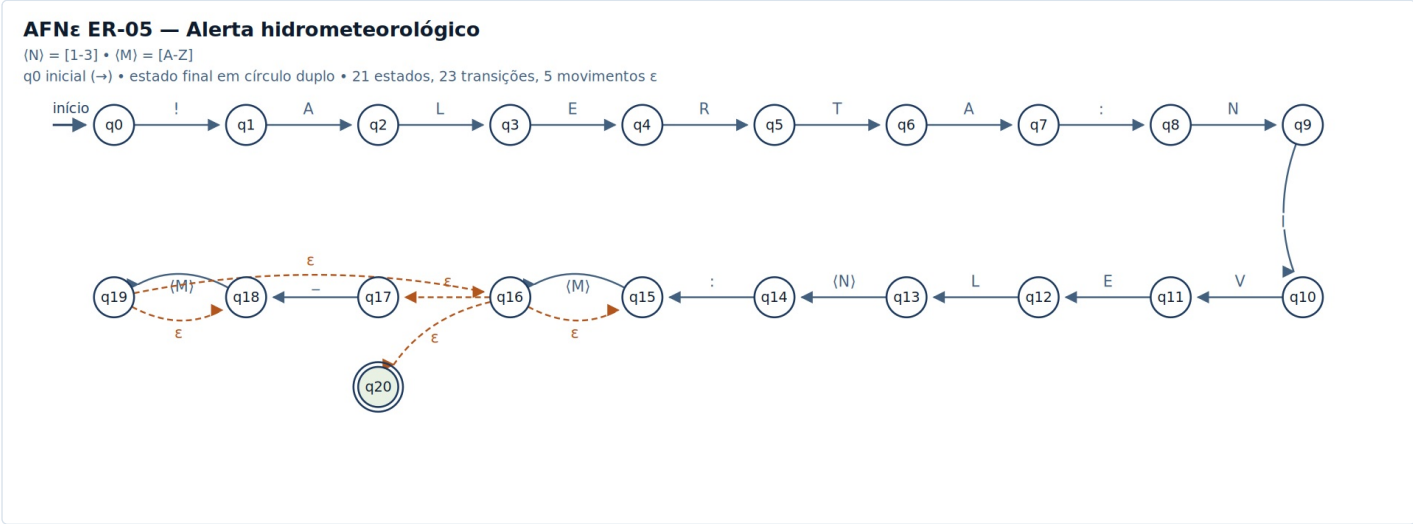
Equivalência entre atalhos e operadores formais

- '!ALERTA:NIVEL' $\equiv$ concatenação de símbolos literais
- [1-3] $\equiv (1|2|3)$
- [A-Z]+ $\equiv M M^*$ (fecho positivo)
- (?:_[A-Z]+)* $\equiv (_ M M^*)^*$ (fecho de Kleene, inclui ϵ)

Operadores utilizados

- concatenação: prefixo fixo do alerta;
- intervalo [1-3]: união dos três níveis de severidade;
- fecho positivo +: cada palavra tem ao menos uma letra;
- fecho de Kleene *: zero ou mais palavras adicionais, o que inclui a palavra vazia ϵ (código de uma só palavra).

AFN ϵ correspondente



Estado inicial: q0 • estado(s) final(is): q20 • 21 estados, 23 transições, 5 movimentos vazios (arestas tracejadas). Descrição formal completa em docs/afne_formal.md.

Testes

#	Cadeia aceita	Cadeia rejeitada
1	!ALERTA:NIVEL1:CHUVA	ALERTA:NIVEL1:CHUVA
2	!ALERTA:NIVEL2:CHUVA_FORTE	!ALERTA:NIVEL4:CHUVA
3	!ALERTA:NIVEL3:VENDAVAL_COM_DESCARGAS	!ALERTA:NIVEL2:chuva
4	!ALERTA:NIVEL3:A	!ALERTA:NIVEL2:
5	!ALERTA:NIVEL2:MARE_ALTA_DE_SIZIGIA	!ALERTA:NIVEL2:CHUVA_
6	!ALERTA:NIVEL1:CALOR	!ALERTA:NIVEL2:CHUVA FORTE

Caso-limite. "!ALERTA:NIVEL2:" — prefixo completo e código vazio: o fecho positivo [A-Z]⁺ impede a cadeia vazia, logo há rejeição. O caso-limite aceito é "!ALERTA:NIVEL3:A", com o menor código possível (uma única letra e o fecho de Kleene em ϵ).

Resultado observado e limitações. Qualquer código de evento em maiúsculas é aceito, inclusive códigos inexistentes no catálogo da Defesa Civil. O programa compara o código com um catálogo interno e marca como DESCONHECIDO quando não o encontra.

ER-06 — Versão de firmware da estação

Finalidade. Validar a versão do firmware embarcado e distinguir versões estáveis de versões de pré-lançamento no relatório de manutenção.

Aplicada em. campo 5 do registro de telemetria

Alfabeto (Σ)

$$\Sigma_6 = D \cup \{ 'f', 'w', 'v', '-', '.', 'a', 'l', 'p', 'h', 'b', 'e', 't', 'r', 'c' \}$$

Linguagem reconhecida (L)

Cadeias iniciadas pelo prefixo literal "fw-v", seguidas de três números inteiros separados por ponto (maior.menor.correção) e, opcionalmente, de um rótulo de pré-lançamento formado por '-', uma das palavras alpha, beta ou rc, um ponto e um número.

Expressão Regular na notação formal

$$f_{w-v} D D^* (\cdot D D^*)^{\{2\}} (- (\alpha | \beta | r c) \cdot D D^* | \varepsilon)$$

Sintaxe exatamente como no código-fonte (Python)

```
r"fw-v[0-9]+(?:\. [0-9]+){2}(?:-(?:alpha|beta|rc)\. [0-9]+)?"
```

Equivalência entre atalhos e operadores formais

- 'fw-' ≡ concatenação de símbolos literais
- [0-9]+ ≡ D D*
- (?:\.[0-9]+){2} ≡ (. D D*) (. D D*)
- (?:alpha|beta|rc) ≡ união de três cadeias literais
- (?:...)? ≡ (... | ε)

Operadores utilizados

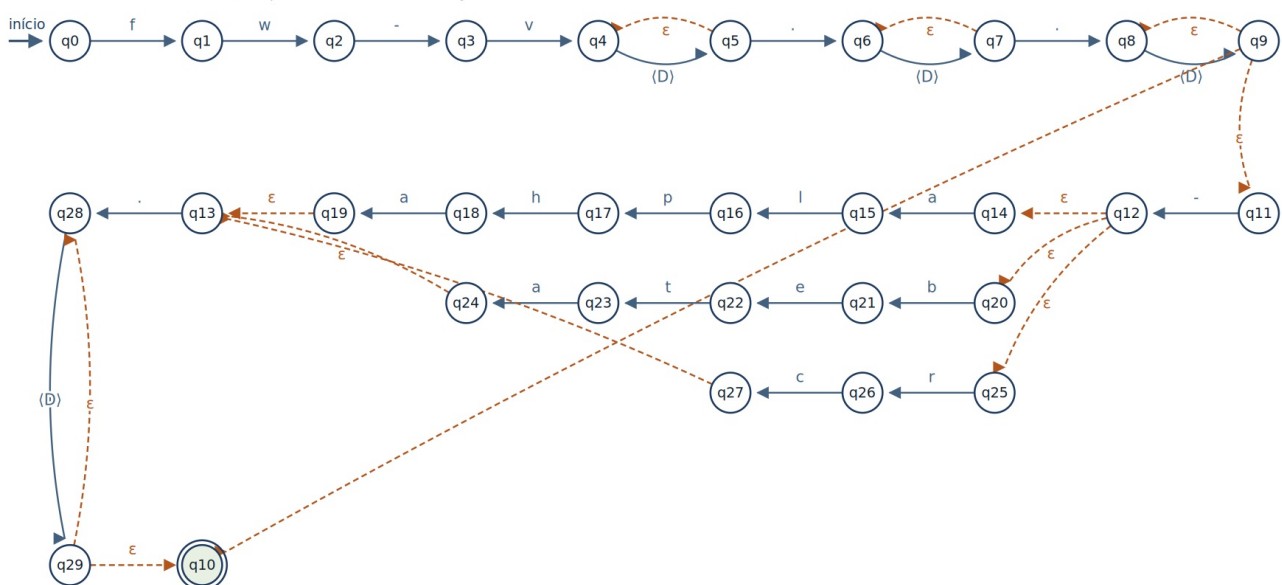
- concatenação: prefixo e blocos numéricos;
- fecho positivo +: cada número tem ao menos um dígito;
- repetição exata {2}: duplica o bloco '.número';
- união |: seleciona o tipo de pré-lançamento;
- opcionalidade ?: versões estáveis não possuem rótulo (ramo ϵ).

AFNε corrispondente

AFNε ER-06 — Versão de firmware da estação

(D) = [0-9]

q0 inicial (→) • estado final em círculo duplo • 30 estados, 36 transições, 13 movimentos e



Testes

#	Cadeia aceita	Cadeia rejeitada
1	fw-v3.11.2	fw-v3.11
2	fw-v0.0.1	fw-v3.11.2.4
3	fw-v12.0.45	v3.11.2
4	fw-v3.11.2-beta.4	fw-v3.11.2-beta
5	fw-v2.7.0-rc.1	fw-v3.11.2-gamma.1
6	fw-v1.0.0-alpha.12	FW-V3.11.2

Caso-limite. "fw-v3.11" — versão com apenas dois componentes: a repetição exata {2} exige dois blocos '.número' após o primeiro número, e a cadeia é rejeitada. O caso-limite aceito é "fw-v0.0.1", a menor versão possível, com um dígito em cada bloco e sem rótulo.

Resultado observado e limitações. A ER não compara versões: "fw-v0.0.1" e "fw-v12.0.45" são igualmente aceitas. A ordenação por recência é feita convertendo os blocos em inteiros após o reconhecimento.

6. Testes e análise dos resultados

Os testes são automatizados com o módulo unittest da biblioteca padrão e podem ser executados com `python -m unittest discover -s testes -t .` (também são reconhecidos pelo pytest). No total são **78 testes**, organizados em três arquivos.

Arquivo	O que verifica
testes/test_expressoes.py	as 6 cadeias aceitas e as 6 rejeitadas de cada ER; os casos-limite isolados; a rejeição da palavra vazia; a ancoragem por fullmatch; a ausência de retroreferências, lookahead, recursão e condicionais; a completude das fichas.
testes/test_automatos.py	a estrutura de cada AFNε (estado inicial e finais pertencentes a Q, alcançabilidade de todos os estados, presença de movimentos ε) e a equivalência ER ↔ AFNε .
testes/test_processador.py	a extração correta dos campos, as verificações semânticas, o tratamento de entradas vazias e inválidas e os códigos de retorno da interface.

6.1 Verificação da equivalência

O teste central compara, para cada uma das seis linguagens, o veredito da Expressão Regular com o do AFNε:

- nas 72 cadeias de teste do projeto, aplicadas *cruzadamente* às seis expressões (432 comparações);
- em 2 000 cadeias aleatórias por expressão, sorteadas sobre o alfabeto do próprio autômato (12 000 comparações);
- em 200 mutações por cadeia aceita — troca, remoção e inserção de símbolos — o que explora justamente a fronteira da linguagem (7 200 comparações).

Não houve nenhuma divergência. A semente do gerador aleatório é fixa, de modo que o resultado é reproduzível.

6.2 Resultado da execução sobre os dados de exemplo

O arquivo `dados/telemetria_setembro.log` contém 36 linhas, das quais 8 são comentários ou linhas em branco. Das 28 linhas de telemetria, **20 foram aceitas e 8 rejeitadas** — estas últimas propositalmente construídas para exercitar, uma a uma, as falhas típicas de cada campo.

<

Os oito registros rejeitados cobrem: código em minúsculas (ER-01), separador de data e hora incorreto (ER-02), coordenada com casas decimais insuficientes (ER-04), leitura sem unidade (ER-03), versão de firmware incompleta (ER-06), nível de alerta fora de [1-3] (ER-05), número de campos insuficiente e data inexistente no calendário — esta última *aceita* pela Expressão Regular e barrada apenas na verificação semântica, o que ilustra de forma concreta o limite do modelo regular.

6.3 Análise crítica

- as seis linguagens são regulares e todas se mostraram implementáveis com os operadores estudados, sem necessidade de recursos avançados do motor;
- os casos-limite concentram-se, como esperado, nas fronteiras dos fechos: ramo ϵ ausente, fecho positivo com zero ocorrências e repetição $\{m, n\}$ fora do intervalo;
- a maior fonte de falsos positivos é a ausência de restrição semântica: datas, faixas geográficas e faixas físicas precisam de verificação numérica posterior;
- o AFN ϵ da ER-02 é o maior (37 estados) por causa dos blocos de tamanho fixo da data e da hora; o da ER-03 é o que mais usa movimentos vazios (19), efeito direto da união de cinco unidades de medida.

7. Limitações e melhorias possíveis

Limitação	Melhoria possível
As ER validam forma, não significado (datas impossíveis, coordenadas fora do planeta, temperaturas implausíveis).	Manter a validação semântica como camada separada e explícita — como já é feito — e ampliá-la com regras por estação.
A estrutura do registro completo é tratada por divisão de cadeia, não por uma única ER.	Compor uma ER do registro inteiro a partir das seis, mantendo-as como subexpressões nomeadas; o AFNε resultante, porém, ficaria muito grande para ser apresentado.
O simulador de AFNε percorre toda a lista de transições a cada símbolo.	Indexar δ por (estado, rótulo) e converter o AFNε em AFD por construção de subconjuntos, exibindo a tabela de estados.
Saída apenas em texto no terminal.	Exportar o relatório em CSV/JSON e gerar gráficos de série temporal por estação.
Unidades limitadas a cinco (C, %, mm, hPa, m/s).	Parametrizar o catálogo de grandezas e unidades em arquivo de configuração, gerando a união da ER-03 dinamicamente.

8. Declaração de uso de Inteligência Artificial

Conforme o enunciado, a equipe declara o uso de assistente de IA generativa nas tarefas abaixo. O uso não substitui a autoria: todo o conteúdo foi revisado, testado e é compreendido pelos integrantes, que sabem explicá-lo e modificá-lo.

Tarefa	Uso da IA	Responsabilidade da equipe
Estruturação dos módulos Python	sugestão de organização em pacotes e funções	revisada, alterada e testada pela equipe
Redação de trechos da documentação e do relatório	apoio de redação e formatação	conteúdo técnico conferido e corrigido
Script de desenho dos diagramas	apoio na escrita do algoritmo de layout	diagramas conferidos manualmente contra as ER
Ampliação da lista de casos de teste	sugestão de cadeias adicionais	todos os casos verificados manualmente

Não houve uso de IA na definição do tema, na escolha das linguagens regulares, na construção conceitual dos AFNε nem na interpretação dos resultados.

9. Contribuição de cada integrante

Integrante	Principais contribuições
[NOME COMPLETO DO INTEGRANTE 1]	ER-01 e ER-02; módulo <code>expressoes.py</code> ; documentação e relatório técnico.
[NOME COMPLETO DO INTEGRANTE 2]	ER-03 e ER-04; módulo <code>processador.py</code> ; dados de exemplo.
[NOME COMPLETO DO INTEGRANTE 3]	ER-05 e ER-06; módulo <code>automatos.py</code> ; diagramas dos AFNε.
[NOME COMPLETO DO INTEGRANTE 4]	módulos <code>cli.py</code> e <code>relatorio.py</code> ; testes automatizados; apresentação.

O registro detalhado, inclusive do roteiro de apresentação por integrante, está em `docs/contribuicoes.md`.

10. Referências

- HOPCROFT, J. E.; MOTWANI, R.; ULLMAN, J. D. *Introdução à teoria de autômatos, linguagens e computação*. 3. ed. Rio de Janeiro: Elsevier, 2008.
- SIPSER, M. *Introdução à teoria da computação*. 2. ed. São Paulo: Cengage Learning, 2007.
- MENEZES, P. B. *Linguagens formais e autômatos*. 6. ed. Porto Alegre: Bookman, 2011.
- THOMPSON, K. Regular expression search algorithm. *Communications of the ACM*, v. 11, n. 6, p. 419–422, 1968.
- PYTHON SOFTWARE FOUNDATION. *re* — *Regular expression operations*. Documentação oficial do Python 3. Disponível em: docs.python.org/3/library/re.html.
- Material de apoio da disciplina: *Guia de Sintaxe para Apresentação das Expressões Regulares*, Trabalho do 1º Bimestre.